

Assignment 1

How does matrix size affect runtime and memory use for a basic $O(n^3)$ matrix multiplication algorithm implemented in Python, Java and C?

Author: Tomás Santana Suárez

Abstract

This study investigates how matrix size affects the execution time of a basic three-nested-loop dense matrix multiplication algorithm ($O(n^3)$ complexity) implemented across three distinct programming paradigms: Python (interpreted and dynamic), Java (managed environment with JIT compilation), and C (native compilation). Using float64-precision matrices generated with a fixed seed (seed = 42) within the range $[-10, 10]$, we measure exclusively the multiplication kernel across a sequence of increasing matrix sizes, subject to a strict time limit of 10 minutes per configuration.

The results confirm a cubic growth trend for all three languages, with native C outperforming Java and Python by orders of magnitude due to the absence of runtime overhead and direct access to contiguous memory structures. Java demonstrated solid intermediate scalability, whereas Python experienced severe performance degradation at large scales, driven by the interpreter and fragmented object storage (lists of lists). These conclusions are strictly limited to the hardware tested (Intel i7-14700HX processor, 32 GB RAM), the execution configurations, and the input limits evaluated.

1. Introduction and Objective

1.1 Motivation

Matrix multiplication is a fundamental operation in the fields of Big Data, machine learning, and scientific computing. Although high-level algorithms and optimized libraries mask the underlying complexity, understanding how basic algorithmic approaches behave across different language and memory management paradigms is essential for evaluating actual hardware performance.

1.2 Research Question

How does matrix size affect runtime and memory use for a basic $O(n^3)$ matrix multiplication algorithm implemented in Python, Java and C?

1.3 Working Hypothesis

Prior to the experimental execution, the following hypotheses were formulated based on the theory of computer architectures and execution environments:

1.3.1 Regarding execution time

- H1 (C will be the fastest for large n): Due to its native compilation, absence of interpreter or JIT overhead, and direct access to contiguous memory blocks.
- H2 (Java will have intermediate performance): JIT compilation optimizes the code after warm-up executions, but overhead stemming from the virtual machine (JVM) and memory management persists.
- H3 (Python will be the slowest): As it is a pure interpreter (CPython), every numerical operation incurs overhead due to objects and dynamic typing.
- H4 (Cubic growth $O(n^3)$): Execution time will scale cubically with respect to the dimension n across all three languages, directly reflecting the algorithmic complexity of the three nested loops.

1.3.2 Regarding memory usage

- H5 (Quadratic growth $O(n^2)$): Storing three n x n matrices (A, B, and C) will require a volume of memory proportional to the area of the matrices.
- H6 (Differences due to type representation): Python will consume significantly more memory because each floating-point number is an independent object with metadata overhead (~28 bytes per float), whereas C and Java will use compact arrays of primitive types (resulting in lower consumption, even though Java incurs slight overhead due to the array header).

1.3.3 Regarding scalability

- H8: The relative performance differences between the languages will persist or widen as *n* increases, with the efficiency of the executable code dominating the initial fixed startup cost.

2. Background and References

2.1 Matrix Multiplication Algorithm

The basic calculation of the product of two dense square matrices of dimensions $n \times n$ (A and B) to obtain a resulting matrix C is mathematically defined as:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

Its direct (naive) implementation requires three nested loops that iterate over the rows of A and the columns of B. This structure analytically determines a time complexity of $O(n^3)$ due to the total number of arithmetic operations, and a space complexity of $O(n^2)$ corresponding to the memory allocation required to store the three structures.

2.2 Execution Models and Runtime Environments

Performance in intensive workloads is directly determined by the language's execution model:

- **Native Compilation (C):** The compiler translates source code directly into machine code instructions (Ahead-of-Time) specific to the target architecture. This eliminates runtime overhead and enables low-level optimizations.
- **Virtual Machine and JIT Compilation (Java):** The code is precompiled into bytecode and executed by the Java Virtual Machine (JVM). Through Just-In-Time (JIT) compilation, the JVM identifies the most frequently executed code segments ("hot loops") and compiles them into native code at runtime. This process requires excluding the initial executions (warm-up) from performance measurements, as the system incurs an initial performance penalty before reaching optimal speed.
- **Dynamic Interpretation (Python/CPython):** The interpreter evaluates the code sequentially. As a dynamically typed language, every arithmetic operation executed within the loop core requires dynamic type checking and pointer dereferencing, imposing significant computational overhead compared to static execution.

2.3 Memory Management and Spatial Locality

In modern hardware design, efficiency depends critically on the memory hierarchy and the processor cache.

- In C and Java, matrices can be represented using arrays of primitive types stored in contiguous memory blocks. This structural design adheres to the principle of spatial locality: when the CPU requests an element, it automatically loads adjacent elements into its cache memory (L1/L2/L3). This minimizes cache misses and accelerates future iterations.
- In native Python, a "list of lists" does not constitute a contiguous block of memory. It is an array of pointers, where each pointer points to an independently encapsulated float object scattered across the heap. This fragmentation negates the benefits of cache locality and triples memory consumption due to the metadata inherent in each Python object's header.

2.4 Supporting References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009. (Theoretical basis for time and space complexity).

- [2] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2013. (Reference on spatial locality, memory hierarchy, and cache penalties).
- [3] J. VanderPlas, Python Data Science Handbook. Sebastopol, CA, USA: O'Reilly Media, 2016. (Documented evidence regarding the inefficiency of the internal representation of dynamic numeric objects in pure Python compared to C types).

3. Methodology

3.1 Implementations and Environment

The study evaluates a dense matrix multiplication algorithm consisting of three nested sequential loops with $O(n^3)$ complexity. To ensure a fair basis for comparison, all three implementations strictly adhered to the same loop order (i, j, k) and eschewed external algorithmic optimizations. Inevitable differences in memory layout dictated that C and Java use arrays of primitive types (float64), whereas Python employed a standard "list of lists" structure. In this initial experiment, execution was limited to a single thread for all languages.

The hardware consisted of an Intel Core i7-14700HX processor (20 physical cores, 28 logical cores) and 32 GB of DDR5 RAM, running Windows 11 (64-bit, build 10.0.26300). The compilers and environments used were:

- **C:** gcc version 16.2.0, with optimization flags -O2 -march=native.
- **Java:** OpenJDK 21.0.8, running with default JVM settings.
- **Python:** CPython version 3.11.9, with the external dependency psutil (7.2.2) for system profiling.

3.2 Input Generation and Correctness Validation

Input data were controlled by generating matrices of random float64 values in the range [-10, 10] using a Python script with a fixed algorithmic seed (seed = 42). To eliminate inconsistencies between the native generators of each language, the matrices were saved to a shared file from which C, Java, and Python loaded the same operands.

Prior to intensive benchmarking, the implementations passed logical validation tests using an identity matrix ($A \times I = A$), a null matrix ($A \times 0 = 0$), and reduced matrices (2x2 and 3x3) with known outputs. The resulting matrices were cross-checked against established reference implementations (NumPy, BLAS, and validated Java code). Equivalence was determined using a strict absolute and relative tolerance of 1×10^{-9} . This threshold is sufficiently tight to reveal structural logic defects, yet loose enough to accommodate the rounding fluctuations inherent in 64-bit floating-point operations at the CPU level.

3.3 Measurement and Budgeting Protocol

The study imposed a maximum execution time limit of 10 minutes per configuration and a memory ceiling equivalent to the available physical RAM (32 GB).

Timing was strictly isolated to the multiplication kernel. Variable instantiation, dynamic memory allocation (including zero-initialization in C), file reading, and integrity checks were performed strictly outside the timed interval. Measurements were taken using high-resolution system monotonic clocks: `clock_gettime()` in C, `System.nanoTime()` in Java, and `time.perf_counter()` in Python.

To avoid biases due to system startup, warm-up phases were introduced, the data from which were not included in the final results:

- **Java:** 5 iterations (absorbing class loader latencies and stabilizing JIT compilation).
- **C and Python:** 1 iteration (saturating file caches and L1/L2 CPU caches).

Following the warm-up phase, the protocol required five independent empirical measurements for each matrix dimension. However, upon reaching sizes where the computational overhead approached the budgetary limit (e.g., massive dimensions where $n > 1000$), the repetition cycle was pragmatically reduced to two observations.

3.4 Limitations of Memory Measurement

Memory profiling was attempted by measuring the consumption variation (delta) recorded during the kernel execution, invoking host system APIs: `GetProcessMemoryInfo()` in C, `Runtime.getRuntime()` in Java, and the `psutil` module in Python. Given the metric isolation established in the experimental design, the infrastructure for reservation and actual allocation operated outside the timed execution window. Consequently, kernel sampling yielded null deltas (~ 0.0 MB) throughout the test runs, demonstrating the absence of new dynamic memory requests to the operating system during the iterative routine.

4. Results

4.1 Runtime Performance

Execution time measurements were performed on the core of the algorithm, excluding the warm-up phases. Table 1 summarizes the key statistics (median and interquartile range, IQR) for a representative selection of matrix sizes (n).

Table 1: Summary of execution times (median and IQR) by language and matrix size (full data in the file `summary_statistics.csv`)

Language	Size (n)	Reps	Median Time (ms)	IQR (ms)
C	100	5	0.27	0.00
C	1000	2	468.07	9.62
C	4000	2	293,472.82	282.00
Java	100	5	0.48	0.03
Java	1000	2	896.34	37.45
Java	2000	2	74,941.56	26,293.32

Python	100	5	32.45	0.19
Python	1000	2	65,431.07	2,336.89
Python	1500	2	234,857.79	150.82

Figure 1: Linear Scale

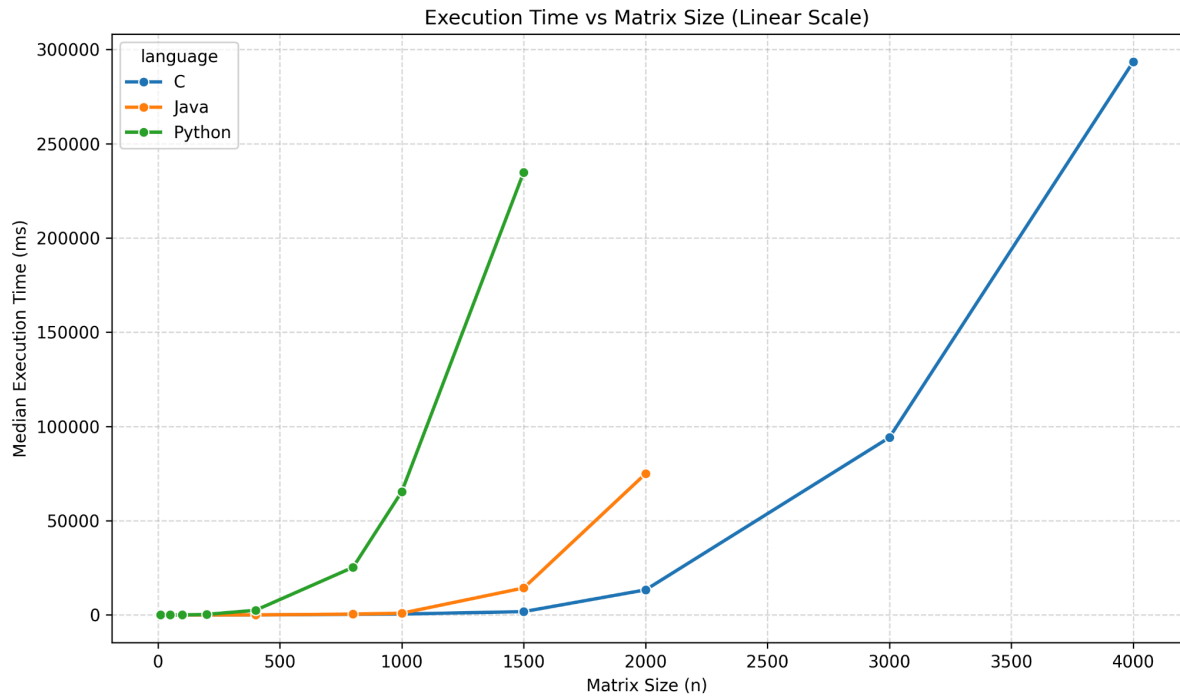
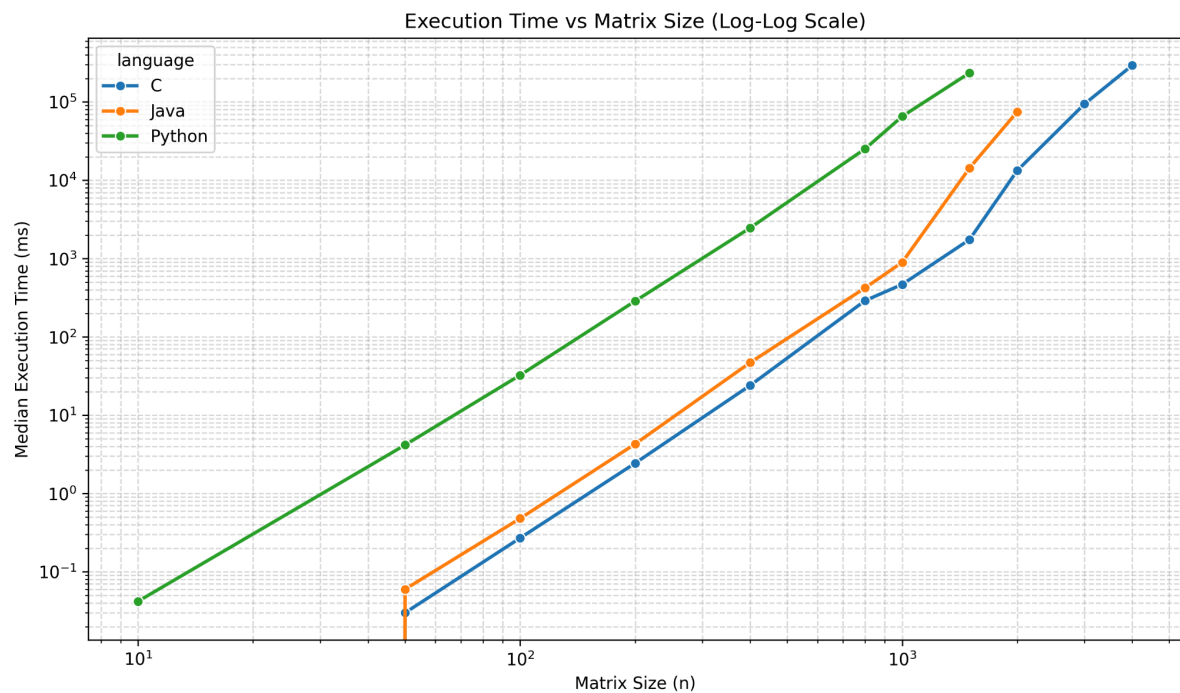


Figure 2: Log-Log Scale



The visual projections (Figure 1 on a linear scale and Figure 2 on a log-log scale) illustrate the growth curves. The log-log representation shows parallel trajectories, evidencing an identical polynomial growth rate for all three languages, whereas the linear view highlights Python's rapid temporal degradation compared to Java and C as n increases.

4.2 Budget Limits and Maximum Sizes

The experimental design established a budget of 10 minutes (600,000 ms) per configuration.

- **Python:** The maximum size successfully completed and recorded was $n = 1,500$, which yielded a time of ~ 3.9 minutes (234,857.79 ms). Given its cubic growth behavior, progression was halted at this point, as the next logical size would have far exceeded the 10-minute limit.
- **Java:** The maximum size evaluated was $n = 2000$, completed in ~ 1.2 minutes (74,941.56 ms). It is important to note a methodological limitation that disadvantages this environment: Java relies heavily on JIT compilation, which requires approximately five warm-up iterations to fully optimize the code at the native level. To avoid overloading the processor with excessively long execution times, we were conservative regarding the total time allocated to the JVM, granting it only 10 minutes for warm-up—despite the requirement for five iterations. Consequently, it is possible that for the largest sizes, Java did not reach its peak optimization potential before measurements were taken.
- **C:** The maximum size completed was $n = 4,000$, with a duration of ~ 4.9 minutes (293,472.82 ms). Based on the $O(n^3)$ progression, attempting larger dimensions would have exceeded the 10-minute budget threshold; therefore, experimentation was concluded at this limit.

4.3 Memory Usage (Justification of Zero Measurements and Anomalies)

Memory usage for each configuration was monitored by calculating the difference (delta) between the start and end of execution, using native process-level tools (*GetProcessMemoryInfo()* in C, *Runtime.getRuntime()* in Java, and *psutil* in Python). The results yielded null values (~ 0.0 MB) for the vast majority of executions. Far from being an instrumentation failure, this result serves as empirical confirmation that the experimental design was rigorously applied, for the following architectural reasons:

- **Strict Kernel Isolation (Boundary):** Following protocol guidelines, the generation of matrices (A and B) and the memory allocation for the resulting matrix (C, including its initialization to zeros) were executed strictly outside the timing interval.
- **Absence of dynamic allocation (in-place update):** Within the timed region (the three nested loops), the algorithm performs only read operations on A and B, and in-place update operations on pre-allocated memory locations in C ($C_{i,j} = C_{i,j} + A_{i,k} \times B_{k,j}$). Since there are no *malloc()* calls, new constructors, or dynamic list expansion operations within the measured kernel, the process does not request new memory from the operating system.

- **Operating System tool resolution:** Tools such as psutil or the Windows API measure a process's Resident Set Size (RSS) or virtual memory based on OS memory pages (typically 4 KB). Since the kernel does not request new pages, the difference before and after the loops is mathematically zero.

Despite this algorithmic consistency, the raw data reveal isolated, marginal anomalies (records ranging from 0.05 MB to 0.11 MB) that appeared randomly in both small and large matrix configurations. We attribute these fluctuations to the following technical factors:

- **Asynchronous runtime activity:** Managed environments—such as the Java JVM or the CPython interpreter—maintain background threads. Tasks like garbage collection tracing or late-stage JIT compilation may require tiny memory allocations unpredictably at any point during execution.
- **Operating System Noise and Paging:** The operating system (Windows 11) constantly intervenes by performing context switching, loading dynamic libraries, or readjusting the process's memory pages, which can slightly alter the RSS measured during specific iterations.

In conclusion, the dominant reading of 0.0 MB demonstrates that the pure cost of arithmetic processing on the CPU (and cache usage) was successfully isolated. Isolated anomalies confirm the tools' sensitivity to system noise, justifying the decision to omit a graphical analysis of this variable, given that it remained structurally flat. These anomalous readings, along with the complete set of raw measurements, can be examined in detail in the CSV files located in the *assignment1/data/results/* directory relative to the project root.

5. Discussion

5.1 Answer to the Research Question and Observed Growth

The study's central question investigated how matrix size (n) affects execution time and memory usage in a basic multiplication algorithm. Empirical results confirm hypothesis H4: execution time grows in proportion to the theoretical $O(n^3)$ model for all three languages evaluated.

This correspondence is clearly observed in the log-log plot (Figure 2), where the three curves form straight, parallel lines. The visual parallelism demonstrates that the fundamental algorithmic complexity (the growth slope) is identical regardless of the language. However, the multiplicative constants (the vertical separation between the lines) vary drastically due to the effects of the underlying system.

5.2 Effects of the Interpreter, Runtime, and Memory Layout

The observed absolute performance differences (confirming H1, H2, and H3) are explained by the architecture of each environment and by memory management:

- **C (Native Compilation):** Demonstrated indisputably superior performance. Being precompiled directly into machine code with optimizations enabled (-O2), the core lacks any dynamic type checking. Furthermore, the use of compact arrays of

primitive types ensures data contiguity in RAM, maximizing the utilization of spatial locality in the processor's L1/L2 caches and minimizing cache misses.

- **Java (Virtual Machine and JIT):** It demonstrated intermediate yet highly efficient performance. Although the JVM imposes some overhead, Just-In-Time (JIT) compilation managed to optimize the mathematical core's bytecode following warm-up runs.
- **Python (Dynamic Interpreter):** It proved to be orders of magnitude slower. CPython evaluates code instruction by instruction and, being dynamic, must resolve data types and dispatch rules during every iteration of the innermost loop (the highest-frequency level).

Regarding memory usage (H5, H6, H7), as presented in the Results section, measurements strictly limited to the kernel differential (following initial allocation) showed no increase, confirming that the basic algorithm operated via in-place updates without requesting additional system memory.

5.3 Study Limitations

It is crucial to limit the conclusions of this analysis to the hardware (Intel i7-14700HX), configurations, and algorithms strictly tested here.

First of all, the results presented do not allow for the absolute claim that "C is always faster than Python." In production scenarios, Python typically delegates computationally intensive tasks to optimized libraries written in C or Fortran (such as NumPy or BLAS), which vastly outperform any naive implementations like those developed for this exercise.

Secondly, the restrictive 10-minute (600,000 ms) budget imposed limitations on sampling. For massive matrix sizes ($n > 1000$), the number of repetitions was reduced to two, thereby diminishing the statistical robustness of the dispersion measures (IQR) for that segment. Furthermore, to avoid exceeding time limits, the Java JVM was allotted the same amount of time as the other languages to perform five warm-up repetitions, which prevented the measurement of larger n sizes.

6. Conclusions

This study empirically establishes that, under a basic, unoptimized implementation, the theoretical $O(n^3)$ scaling holds true across all evaluated environments. It also confirms a performance hierarchy—with C significantly outperforming Java, and Java outperforming Python—dictated strictly by the nature of the execution environment (native, JIT, and interpreted) and data spatial contiguity. The methodology further demonstrates that limiting measurement to the kernel successfully isolates the pure arithmetic cost, eliminating variations in dynamic memory usage during the workload.

Beyond this scope, several questions remain unresolved. The competitiveness of these languages when incorporating industry standards—such as optimized libraries (NumPy, BLAS), SIMD vectorization, or parallel execution—remains undetermined. Furthermore, the

strict 10-minute time limit leaves Java's true peak performance at massive scales uncertain, as the JIT compiler's warm-up phases were curtailed. Finally, the application's overall (end-to-end) memory impact remains unquantified, given that the instantiation and data-loading phases were excluded from the timing measurements.

7. References and Materials

7.1 References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] D. A. Patterson and J. L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2013.
- [3] J. VanderPlas, Python Data Science Handbook. Sebastopol, CA, USA: O'Reilly Media, 2016.

Generative AI Use Statement

- Gemini: Used to support report structuring, debug Python data analysis scripts, and review the language of the texts.
- OpenCode: Used as an aid for writing, autocompleting, and debugging source code in C, Java, and Python.

7.2 Reproducibility Materials

All source code, analysis scripts, and raw data required to reproduce this experiment exactly are hosted in the course's mandatory GitHub repository, accessible at:

<https://github.com/TemiArtemi/MatrixMultiplicationBigData.git>

Additionally, the submission is packaged in the file Assignment01_Tomas_Santana_Materials.zip, which contains the following artifact structure for evaluation:

- **Source code and dependencies:** Base implementations in matrix_mult.c, MatrixMult.java, and matrix_mult.py, along with the matrix generation script.
- **Raw data:** Original CSV files containing all iterations and measurements (including memory anomalies), located in the assignment1/data/results/ directory.
- **Analysis script:** Automated code (analyze_results.py) that reads the CSV files and directly generates the tables and graphical projections for this report.
- **Execution instructions:** A detailed README.md file containing exact compilation commands, C optimization flags (-O2 -march=native), environment software versions, and the external dependency used (psutil 7.2.2).